

Subject: Exploring Advanced SQL Topics for Review

Review Date: 5 Jan

Reviewer: Sanjay Sanjeevan

Advanced SQL Concepts

1. Data Reconciliation in SQL

What is Data Reconciliation?

Data reconciliation is the process of comparing data from two or more sources to ensure consistency, accuracy, and completeness.

Why it is Important

- Ensures data accuracy between source and target systems
- Detects missing, duplicate, or mismatched records
- Critical in ETL pipelines, financial data, and reporting systems

Common Use Cases

- Source vs Data Warehouse comparison
- Daily transaction count matching
- Financial balance checks

Techniques in SQL

- COUNT comparison
- SUM comparison
- FULL OUTER JOIN to find mismatches

Example

```
-- Count reconciliation
SELECT COUNT(*) FROM source_table;
SELECT COUNT(*) FROM target_table;

-- Data mismatch detection
SELECT s.id, s.amount, t.amount
FROM source_table s
FULL OUTER JOIN target_table t
ON s.id = t.id
WHERE s.amount <> t.amount OR s.amount IS NULL OR t.amount IS NULL;
```

2. Incremental Data Load in SQL

What is Incremental Data Load?

Incremental data loading means loading only new or changed records instead of reloading the entire dataset.

Why it is Important

- Improves performance
- Reduces processing time
- Saves system resources

Common Strategies

- Based on timestamp (CreatedDate / ModifiedDate)
- Based on surrogate key or ID
- Change Data Capture (CDC)

Example

```
INSERT INTO target_table
SELECT *
FROM source_table
WHERE ModifiedDate > (SELECT MAX(ModifiedDate) FROM target_table);
```

Incremental Load Using MERGE

```
MERGE target_table t
USING source_table s
ON t.id = s.id
WHEN MATCHED THEN
UPDATE SET t.amount = s.amount
WHEN NOT MATCHED THEN
INSERT (id, amount) VALUES (s.id, s.amount);
```

3. Data Auditing

What is Data Auditing?

Data auditing tracks changes made to data for accountability, compliance, and debugging.

Why it is Important

- Helps in data governance
- Tracks who changed what and when
- Useful for compliance and rollback

Types of Data Auditing

- Insert audit
- Update audit
- Delete audit

Common Methods

- Audit tables
- Triggers
- Change tracking

Example (Audit Table)

```
CREATE TABLE Employee_Audit (  
    EmpId INT,  
    OldSalary INT,  
    NewSalary INT,  
    ChangeDate DATETIME,  
    ActionType VARCHAR(10)  
);
```

4. Soft Delete

What is Soft Delete?

Soft delete means marking a record as deleted instead of physically removing it from the database.

Why it is Used

- Data recovery
- Historical tracking
- Audit compliance

Common Implementation

- IsDeleted flag
- DeletedDate column

Example

```
UPDATE Employees  
SET IsDeleted = 1, DeletedDate = GETDATE()  
WHERE EmpId = 101;
```

```
Fetch Only Active Records  
SELECT * FROM Employees  
WHERE IsDeleted = 0;
```

5. SQL Anti-Patterns

What are SQL Anti-Patterns?

SQL anti-patterns are poor database design or query practices that lead to performance, scalability, or maintenance issues.

Common SQL Anti-Patterns

1. Using SELECT *

- Fetches unnecessary columns
- Impacts performance

-- Bad Practice

```
SELECT * FROM Employees;
```

-- Good Practice

```
SELECT EmpId, EName, Salary FROM Employees;
```

2. Storing Multiple Values in One Column

- Breaks normalization

-- Bad

```
Skills = 'Java,SQL,Python'
```

3. Not Using Indexes Properly

- Causes slow queries

4. Overusing Subqueries Instead of Joins

5. Ignoring NULL Handling